

Key tools for cyber security and digital forensics

- Steganography and hidden information extraction
- E-mail fingerprinting
- Image and video forensics
- Remote access denial
- Web scraping
- Internet of Things (IoT) security
- Web applications security
- Vulnerability analytics
- Anti-exploitation tools
- Anti-sniffing and anti-spoofing attacks

Python tools for cyber security and digital forensics

There are a number of programming languages and tools available for cyber security and digital forensic applications, but Python is the most widely used. Python has hundreds of thousands of modules on its official repository *PyPi.org*. These cover assorted domains including cyber security, grid computing, information security, cloud applications, Web scraping, image forensics and many others. Python is one of the most used programming languages by cyber security professionals since it has numerous free and open source tools and packages.

Image EXIF and Python programming

Every digital image has exchange information (EXIF) data that can be used to identify the actual camera or device that captured the image. Python provides many libraries and packages for EXIF metadata to recognise the root of a viral image.

Python integrates the package Python Image Library (PIL) for image analytics and forensic applications. It is also used for the extraction of EXIF metadata to recognise the camera information as well as the location at which the image was taken. The PIL package is available in the new version as Pillow. It can be installed using the PIP installer as follows:

```
$ pip install Pillow
```

```
'Make': 'NIKON', 'Model': 'COOLPIX P1',
'Software': 'GIMP 2.4.5'}
{'ResolutionUnit': 2, 'ExifOffset': 224,
'ImageDescription': ' ',
'Make': 'NIKON', 'Model': 'COOLPIX P1',
'Software': 'GIMP 2.4.5', 'Orientation':
1}
```

Figure 1: Camera information using EXIF in Python

Exif Image Size	200 x267
Image Description	
Make	PANASONIC
Software	MediaTek Camera Application
Modify Date	2019:09:26 10:02:56 3 hours, 59 minutes, 11 second in the future
Exif0x0220~0x0224	0 (5 times)
Exif0x0225	
Exposure Time	1/17
F Number	2.20
Exposure Program	Not Defined
ISO	720
Exif Version	0220
Date/Time Original	2019:09:26 10:02:56 3 hours, 59 minutes, 11 second in the future
Create Date	2019:09:26 10:02:56 3 hours, 59 minutes, 11 second in the future

Figure 2: Smartphone information using EXIF

In addition to image forensics, the following features are available in PIL:

- Pixel based manipulations and forensics
- Image masking
- Image transparency
- Image filtering
- Edge detection
- Image smoothing
- Blurring
- Enhancements
- Steganography

The Python library PIL provides a few lines using which deep information about the image can be extracted in real-time. The following is an example of fetching the image's location using exchange information (EXIF).

```
import PIL
img=PIL.Image.open('viralimage.jpg')
exifdata=img._getexif()
print (exifdata)
```

Nowadays, the location and camera information associated with viral images can be extracted using EXIF data. If a specific image or video is going viral, the first person who took that image from the camera can be identified using EXIF.

As shown in Figure 1, the camera information of the image is recognised using the EXIF code of Python.

After formatting and advanced manipulations in the EXIF, the information about the actual smartphone that captured a particular photograph can be fetched (Figure 2).

The deep analytics using EXIF shows the exact name and model of the smartphone that was used to take the image. By this methodology, the forensics associated with the image is very easy when using Python programming.

Web server fingerprinting

Web server fingerprinting refers to the identification of internal information about a remote server or IP address used for the specific assault. If a specific attack is mounted using a particular system, it can be identified. On attempting any cyber attack, the source system can be deeply investigated using socket programming in Python. Socket programming is used to connect to the remote system in order to analyse the root attacker system.

```
>>> import socket
>>> socket.gethostbyname('URL')
>>> socket.gethostbyaddr('IP Address')
```

Using these inbuilt functions of Python, the URL or IP address can be tracked. In addition, there are assorted libraries in Python that are used to extract deep information about the devices or remote systems.